

Deep Dive — Module 2: Qdrant (Vector Database)

Goal: understand how the project stores and searches meaning rather than exact text — collections, points, vectors, similarity metrics, payload filtering, the HNSW index, and how Qdrant pairs with the embedding service to power similarity search and duplicate detection.

Where Qdrant sits: It's the **semantic search engine**. PostgreSQL stores facts; Qdrant stores the 384-number "meaning fingerprint" of every text chunk and answers "*which chunks are closest to this one?*" in milliseconds. Every similarity search and duplicate-detection feature ultimately bottoms out in a Qdrant query.

1. Why a vector database (vs PostgreSQL)

A relational `WHERE vendor = 'Acme'` finds **exact** matches. It cannot find "this invoice *means* the same as that one, despite different wording." Qdrant solves *semantic* search: text is turned into a **vector** (a point in 384-dimensional space) and similar meanings land close together. Qdrant's whole job is finding the nearest points to a query vector, fast, even across hundreds of thousands of vectors.

2. Vectors & embeddings

A **vector** is just a list of numbers; here, 384 floats produced by the `all-MiniLM-L6-v2` model (Module 5). The model is trained so that *similar meaning* → *similar numbers*:

"Invoice from Acme Corp"	→ [0.023, -0.118, 0.204,...]	└ close together
"Bill from Acme Corporation"	→ [0.021, -0.115, 0.201,...]	└ (similar meaning)
"Bank statement HDFC Q1 2026"	→ [-0.142, 0.307, -0.089,...]	← far away (different topic)

The number **384** is fixed by the model. Qdrant's collection must be configured for exactly 384 dimensions — mismatches are rejected.

3. Cosine similarity (the metric)

Cosine similarity measures the **angle** between two vectors, ignoring their length:

```
1.0 → identical direction (same meaning)
0.0 → perpendicular (unrelated)
-1.0 → opposite direction
```

The project uses `"distance": "Cosine"` because it's robust to text length differences (a short invoice and a long one with the same meaning still match). Qdrant returns a **score** per result; higher = more similar.

Qdrant also supports Euclidean (`Dot` , `Euclid`) distances. Cosine is the standard choice for sentence-transformer embeddings, which are normalized.

4. Collections

A **collection** is Qdrant's equivalent of a table — a named group of vectors that share dimensionality and distance metric. The project uses one: `financial_documents`.

```
curl -X PUT http://localhost:6333/collections/financial_documents \
-H 'Content-Type: application/json' \
-d '{ "vectors": { "size": 384, "distance": "Cosine" }, "on_disk_payload": true }'
```

- `size: 384` — must match the embedding model output exactly.
- `distance: "Cosine"` — the similarity metric.
- `on_disk_payload: true` — keep metadata on disk (saves RAM) while vectors stay in the fast HNSW index.

5. Points (vector + id + payload)

A **point** is one record: an `id`, the `vector`, and a `payload` (arbitrary metadata, like SQL columns living next to the vector).

```
{
  "id": "a-uuid",
  "vector": [0.023, -0.118, 0.204, ... 381 more. ],
  "payload": {
    "document_id": "doc-a3f9c1b2", "filename": "invoice_2048.pdf",
    "vendor": "ABC Supplies", "invoice_number": "INV-2048",
    "invoice_date": "2026-03-15", "total_amount": 1250.75,
    "currency": "USD", "document_type": "invoice", "chunk_index": 0
  }
}
```

Crucial design point: one *document* becomes *many* points — one per text chunk (Module 9). Every chunk-point carries the same `document_id` in its payload so results can later be grouped back to the parent document.

6. Upserting points

```
POST http://localhost:6333/collections/financial_documents/points
{ "points": [ { "id": "...", "vector": [...], "payload": {...} }, ... ] }
```

Upsert = insert if new, overwrite if the id already exists. This makes re-indexing a document idempotent — re-uploading doesn't create duplicate vectors, as long as you reuse deterministic point ids.

7. Searching for nearest neighbours

```
POST http://localhost:6333/collections/financial_documents/points/search
{ "vector": [0.031, -0.102, 0.198, ... ], "top": 5, "with_payload": true }
```

Returns the closest points with scores and payloads:

```
[
  {"id": "pt-1", "score": 0.94, "payload": {"vendor": "ABC Supplies", "filename": "invoice_2048.pdf"}},
  {"id": "pt-7", "score": 0.87, "payload": {"vendor": "ABC Supplies", "filename": "invoice_2031.pdf"}},
  {"id": "pt-3", "score": 0.71, "payload": {"vendor": "Globex Ltd", "filename": "receipt_089.pdf"}}
]
```

Because a query document also has multiple chunks, the backend runs *one search per query chunk*, then **merges results by `document_id`**, taking the max score per document — so the strongest matching chunk represents each document.

8. Payload filtering (vector search + metadata)

You can constrain the search to points whose payload matches conditions — like a SQL `WHERE` applied *during* the nearest-neighbour search:

```
{
  "vector": [0.031, -0.102, 0.198,... ],
  "top": 5,
  "filter": {
    "must": [ {"key": "vendor", "match": {"value": "ABC Supplies"}},
              {"key": "currency", "match": {"value": "USD"}} ],
    "must_not": [ {"key": "document_type", "match": {"value": "statement"}} ],
    "should": [ {"key": "invoice_date", "range": {"gte": "2026-01-01"}} ]
  }
}
```

- `must` = AND (all required), `should` = OR (boost), `must_not` = exclude.
- Supports `match` (exact), `range` (numeric/date), and more.

This powers the search API's `vendor`, `currency`, `dateFrom/dateTo` filters. For payload filtering to be fast at scale, create **payload indexes** on frequently filtered keys (`vendor`, `currency`, `document_type`).

9. HNSW index (why it's fast)

Comparing a query against 500,000 vectors one-by-one is slow. **HNSW** (Hierarchical Navigable Small World) builds a multi-layer graph of "who is near whom" and *navigates* it, checking only a few hundred strategic nodes.

```
Brute force: compare against all 500,000 vectors → seconds
HNSW:       navigate graph, ~200 node checks   → < 50 ms
```

HNSW is **approximate** — it may very rarely miss the true nearest neighbour, traded for huge speed. Tunable params: `m` (graph connectivity) and `ef / ef_construct` (search/build effort). Defaults are fine for this project's scale; raise `ef` for higher recall at some latency cost.

10. Score thresholds & labels (this project)

<code>score ≥ 0.85</code>	→ <code>STRONG_MATCH</code>	(likely duplicate / near-identical)
<code>0.70 - 0.84</code>	→ <code>RELATED</code>	(same vendor/topic, different transaction)
<code>0.60 - 0.69</code>	→ <code>WEAK_MATCH</code>	(show with caution)
<code>< 0.60</code>	→ filtered out	

Special rule: `score ≥ 0.90 AND matching invoice_number` → `CONFIRMED DUPLICATE ALERT`

The backend applies the threshold *after* Qdrant returns scores, then maps scores to these labels for the UI. The duplicate rule combines a *semantic* signal (score) with an *exact* signal (matching invoice number from PostgreSQL) — neither alone is enough.

11. Snapshots (backup) & operations

```
POST http://localhost:6333/collections/financial_documents/snapshots
```

Qdrant writes a `.snapshot` file you store off-server; restore re-creates the collection. Combine with PostgreSQL backups so metadata and vectors stay consistent. For GDPR erasure, deleting a document also issues `DELETE /points` with a filter on `document_id` to purge all its chunk-vectors.

12. How the backend talks to Qdrant

Spring Boot's `QdrantService` calls these REST endpoints (or uses the Qdrant Java client) via `RestTemplate / WebClient`. Qdrant listens on `:6333` (REST) and `:6334` (gRPC). It runs as a Docker container (Module 6) and is **not exposed to the internet** — only the backend on the internal Docker network reaches it.

13. Common pitfalls

- **Dimension mismatch:** collection `size` must equal the model's output (384). Switching to a 768-dim model means re-creating the collection *and* re-embedding everything.
 - **Forgetting `with_payload: true`** : you get ids and scores but no metadata to display.
 - **Not grouping by `document_id`** : chunk-level results show the same document multiple times. Always merge to one row per document (max score).
 - **No payload index on filtered keys:** filtered searches get slow as the corpus grows.
 - **Treating HNSW as exact:** it's approximate; raise `ef` if you need higher recall.
 - **Re-upsert with random ids:** creates duplicate vectors. Use deterministic point ids tied to `document_id + chunk_index`.
-

14. Practice exercises

Run Qdrant locally: `docker run -p 6333:6333 qdrant/qdrant`.

1. Create the `financial_documents` collection (384, Cosine). Verify with `GET /collections/financial_documents`.
 2. Upsert 5 points with small *fake* 4-dim vectors (use a scratch collection of size 4) and payloads for two vendors. Search for the nearest 3 to a query vector and read the scores.
 3. Repeat the search with a `must` filter on one vendor; confirm only that vendor's points return.
 4. Add a `range` filter on `invoice_date` and confirm date filtering works.
 5. Upsert the *same* id twice with different payloads; confirm the point is updated, not duplicated.
 6. Simulate the project flow: insert 3 chunk-points sharing one `document_id`, search, and write pseudo-code to merge results by `document_id` taking the max score.
 7. Map a set of returned scores (e.g. 0.93, 0.82, 0.64, 0.41) to the project's labels.
 8. Create a snapshot, then delete a point by filtering on `document_id`, simulating GDPR erasure.
-

15. Self-check questions

- Why can't PostgreSQL alone do similarity search?
 - What three parts make up a Qdrant point?
 - Why must the collection size equal the embedding model's output dimension?
 - How does the backend turn many chunk-level matches into one result per document?
 - What does HNSW trade away for speed, and how do you tune it?
 - How does the duplicate-alert rule combine Qdrant and PostgreSQL signals?
 - What's the difference between `must` , `should` , and `must_not` filters?
-

16. Glossary

- **Vector / embedding** — list of numbers representing meaning (384 dims here).
 - **Collection** — named group of vectors (Qdrant's "table").
 - **Point** — one record: id + vector + payload.
 - **Payload** — metadata stored with a vector; filterable.
 - **Cosine similarity** — angle-based similarity metric; 1 = identical.
 - **Upsert** — insert-or-update.
 - **HNSW** — graph index for fast approximate nearest-neighbour search.
 - **ANN** — Approximate Nearest Neighbour search.
 - **Snapshot** — Qdrant backup file.
-

Navigation: [← Module 1 PostgreSQL](#) | [Index](#) | [Module 3 React →](#)